Proceedings of the 2026 Winter Simulation Conference
V. Ramamohan, A. Djanatliev, M. Fakhimi, C. Krejci, C. Ruiz Martin, B. S. Onggo, and N. Mustafee, eds.

A TUTORIAL ON GENERATIVE AI AND SIMULATION MODELING INTEGRATION

Mohammad Dehghani, Sahil Belsare, and Negar Sadeghi

Department of Mechanical and Industrial Engineering
Northeastern University
360 Huntington Ave, Boston, MA 02115, USA

ABSTRACT

This tutorial paper explores the applicability of Generative AI (GenAI), particularly Large Language Models, within the context of simulation modeling. The discussion is organised around five stages, each treated as a capability ladder rising in complexity. *Problem Formulation* produces a structured brief that the rest of the workflow consumes. *Creation* turns the brief into a runnable simulation through input modelling, structured-prompt model generation under a validation gate, and an agentic construction route via the Model Context Protocol (MCP). *Execution* narrates trace output, orchestrates run-time activity, and lets the LLM act as an in-simulation decision agent. *Experimentation* ideates scenarios, generates result summaries, and (with explicit caveats) drives optimisation. *Evaluation* closes the loop with reproducibility artefacts, validation against analytical bounds and commercial reference models, and a cross-model benchmark of frontier LLMs. Building on the 2025 edition of this tutorial, the present revision adds the MCP route, a comparison of heuristic versus LLM-in-the-loop decision rules inside a running model, and a fully reproducible repository with an in-browser live demonstration. The tutorial offers both conceptual guidance and hands-on examples to support researchers and practitioners seeking to integrate GenAI into simulation environments.

1 INTRODUCTION

Some of the most revolutionary products appear to be conceived in a single night, yet they are almost always built on decades of quiet, accumulating research. The simulation modelling community knows this story well, and so does the artificial-intelligence community next door. Both fields have followed a strikingly parallel arc over the same six decades. Simulation went through a language era in the early 1960s with GPSS, SIMULA, SLAM, and SIMAN, then a GUI revolution in the 1990s with ProModel and Arena, then the multi-method platforms of the 2000s such as AnyLogic and Simio, and most recently an AI-augmented era that brings neural networks, reinforcement learning, and generative AI directly inside the modelling environment. Artificial intelligence followed the same staircase from a different starting point. The symbolic era ran from Dartmouth in 1956 through MYCIN in 1976. Statistical and neural methods took over from the late 1980s with backpropagation, support-vector machines, and random forests. The deep-learning revolution of the 2010s brought AlexNet and AlphaGo. Generative AI arrived in the early 2020s, with ChatGPT in 2022 as the public turning point (Figure 1). Each wave on each side reshaped what the technology could do. None arrived overnight. The point is not that 2025 was the year of generative AI. It is that 2025 was the year the two timelines started intersecting at the level of practitioner workflows.

The 2025 edition of this tutorial (Dehghani et al. 2025) introduced a 3 by 3 capability ladder for thinking about that intersection in the context of discrete-event simulation: three workflow stages (Generation, Execution, Analysis) along the horizontal axis and three rising-complexity LLM contributions inside each, for nine cells in total. The present paper is the continuation of that tutorial. It is updated for a year in which generative AI has reshaped what is possible at a speed without precedent in computing history. ChatGPT reached one hundred million monthly users within two months of launch, the fastest adoption curve ever recorded for a consumer technology (K. Hu 2023). Frontier models that struggled with discrete-event code

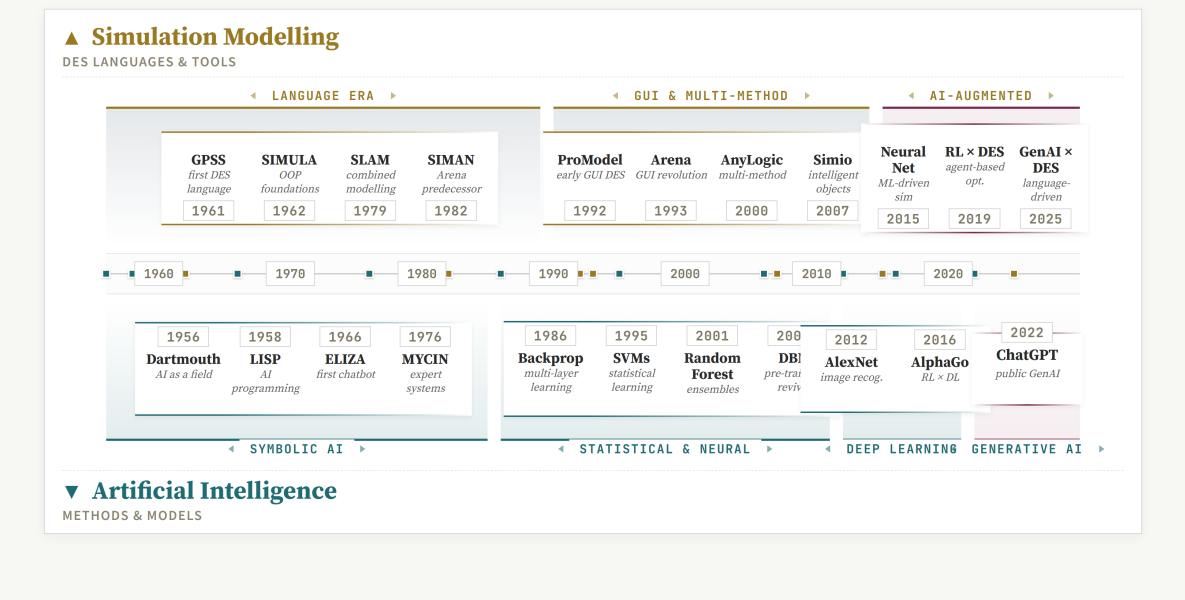


Figure 1: Parallel evolution of simulation modelling and artificial intelligence. The two timelines develop largely independently for half a century and begin to intersect in the practitioner workflow only in the 2020s.

in 2025 now produce working SimPy scripts on the first attempt. The Model Context Protocol (Anthropic 2024) has emerged as a standard way for language models to invoke external tools. And reproducible runnable artefacts, rather than purely conceptual tutorials, are increasingly what the community asks for.

This paper makes the following contributions:

- A revised tutorial that restructures the 2025 framework into a 5 by 3 capability ladder: five workflow stages — Problem Formulation, Creation, Execution, Experimentation, Evaluation — each containing three rising-complexity LLM contributions (Figure 5).
- A working agentic-AI workflow built on the Model Context Protocol that allows language models to construct simulation models by calling exposed simulation primitives, rather than generating monolithic source files.
- An empirical cross-model comparison of two frontier large language models acting as in-simulation decision agents (LLM-in-the-loop bed assignment) against deterministic heuristic baselines, with paired-seed replications and a fully reproducible experiment harness.
- A fully reproducible accompaniment to the paper: a public source-code repository and an in-browser live demonstration of the running example.[1]

Large language models in sixty seconds. Generative AI has reshaped the technology landscape in the four years since ChatGPT's public release in late 2022, and at the centre of that change sits a single architecture, the large language model. Figure 2 places LLMs within the broader AI landscape as a nested hierarchy: Artificial Intelligence is the umbrella, Machine Learning a subset that learns from data, Deep Learning a further subset that uses multi-layer neural networks, Generative AI the slice of deep learning concerned with producing new content, and large language models the most prominent class of generative models, text-generators trained at unprecedented scale. An LLM predicts the next token in a sequence using the Transformer architecture (Vaswani et al. 2017), which replaced the strictly sequential

[1] Source and live demo: https://github.com/mdehghani86/wsc26-genai-sim and https://mdehghani86.github.io/wsc26-genai-sim/.

processing of older recurrent networks with self-attention. Three steps run in order: the input string is tokenized and each token is mapped to a dense vector with a positional encoding (Figure 3a); a stack of Transformer blocks repeatedly applies multi-head self-attention and feedforward layers; and the final layer produces a probability distribution over the vocabulary, from which the next token is sampled (Figure 3b). Three parameters control how much variation that sample tolerates. *Temperature* rescales the distribution before sampling: low values concentrate mass on the most likely token, high values flatten the distribution (Figure 4). *Top-k* restricts the sampling pool to the k highest-probability tokens. *Top-p*, also known as nucleus sampling, keeps the smallest set of top tokens whose cumulative probability exceeds a threshold p, which adapts to the local shape of the distribution. Throughout this paper we run with temperature 0.2 for code-generation prompts and 0.7 for narration prompts, with top-p 0.9 in both cases. The 2025 edition (Dehghani et al. 2025) treats this material at full length and is the recommended primer for readers who want the lower-level view.

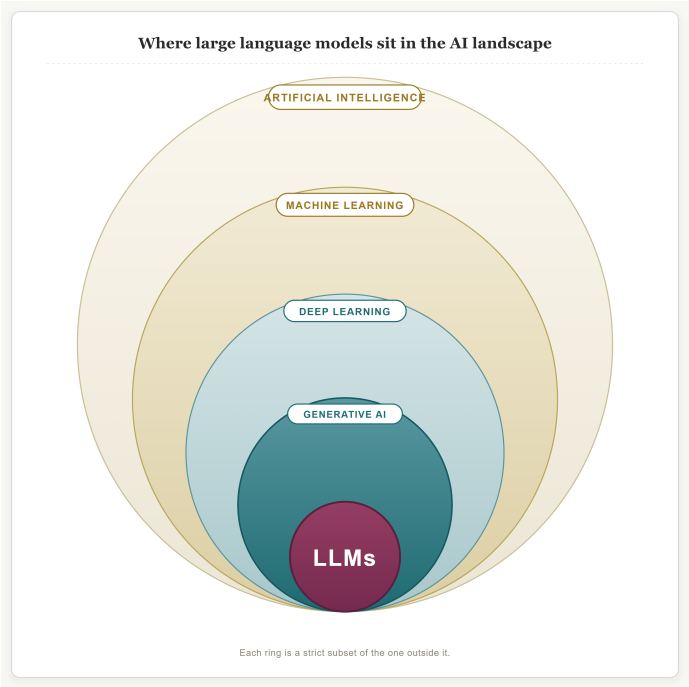
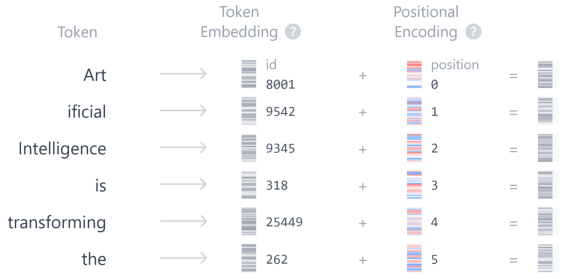
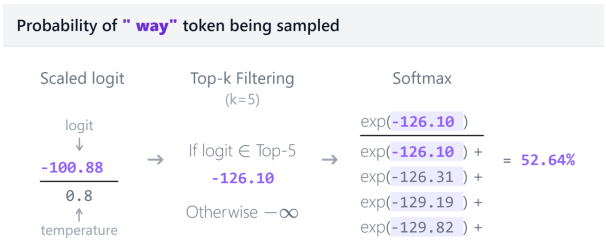


Figure 2: LLMs as the innermost layer of a nested AI hierarchy: AI ⊃ ML ⊃ DL ⊃ GenAI ⊃ LLMs.



(a) Token embedding and positional encoding.



(b) Output probability distribution over the next token.

Figure 3: First and last steps of a Transformer pass.

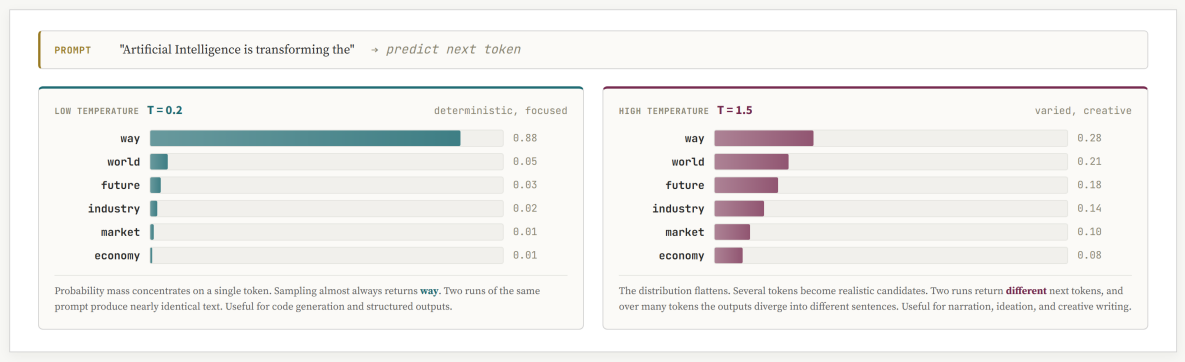


Figure 4: Same prompt under two temperatures: T=0.2 concentrates probability on a single token; T=1.5 flattens the distribution.

Running example: emergency department. A multi-stage emergency department (ED) model is used as a single running case study throughout the paper. Patients arrive at the ED, are triaged into severity bands, may pass through imaging or laboratory services, and are then either admitted to inpatient beds or discharged. The case is intentionally rich enough to expose where current LLMs succeed and where they break: it requires branching, state tracking, multiple resource pools, and quantitative validation against analytical bounds. Two side-by-side comparisons run on this case throughout the paper: vibe coding versus MCP-driven structured construction at the Creation stage (Section 3.3), and heuristic versus LLM-in-the-loop policies at the Execution stage (Section 4.3).

The 5 by 3 workflow. The remainder of the paper organises GenAI-assisted simulation work into five typed stages, each containing three rising-complexity LLM capabilities (Figure 5). The 2025 edition used three stages and described the framework in narrative terms. The 2026 revision adds an explicit upstream stage, Problem Formulation, and a downstream stage, Evaluation, and tightens the contract between stages. Each stage produces a single artefact that the next stage consumes verbatim. Problem Formulation produces a three-row brief (problem, scope, KPIs). Creation produces a validated SimPy file together with its passing test report. Execution produces a run-time trace and the in-simulation decisions taken along the way. Experimentation produces a scenario table together with the figures and prose that interpret it. Evaluation produces the reproducibility artefacts, the validation harness, and the cross-model scorecard. The contract between stages is what gives the workflow its discipline. A figure that does not trace back to a row in the brief is not part of the study. A scenario that the brief did not ask for is not reported. The five-check validation gate at the end of Creation is what makes downstream consumption safe. If the gate does not pass, no downstream stage runs.

2 STAGE 0: PROBLEM FORMULATION

Before any code is written, the modeller has to settle what is being modelled and why. The first stage produces no code and no model file. It produces a written agreement on three items, and along the way uses the LLM in three rising-complexity ways:

- **Idea Capture.** The LLM is prompted in natural language with the study question and produces a candidate framing. This is the simplest contribution — a rephrasing rather than a verdict.
- **Brief Structuring.** The candidate framing is forced into a fixed three-row schema (problem, scope, KPIs) so that downstream stages consume the artefact verbatim.
- **KPI Discovery.** The LLM is asked to consult recent literature and propose KPIs that reflect community practice, rather than the modeller's prior habits, for the third row of the schema.

Figure 5: The 5 by 3 GenAI-driven simulation framework. Five workflow stages (Problem Formulation, Creation, Execution, Experimentation, Evaluation) along the horizontal axis, three rising-complexity LLM capabilities along the vertical axis, for fifteen cells in total. Each stage produces a typed artefact that the next stage consumes. (Figure to be regenerated for the 2026 5 by 3 layout — placeholder shows the 2025 four-stage version.)

A recent analysis of professional GenAI use found ideation to be one of its most common roles (M. Zao-Sanders 2025). What we add here is structure: the conversation is the same, but the output is constrained to a fixed schema so that the artefact can be carried forward unchanged. All prompts shown in this paper were issued to Anthropic's Claude[2] and reproduced verbatim from the chat transcript.

Figure 6 shows the prompt and response for the ED running example. The prompt is doing three things at once. It states the request in natural language. It supplies a short note that anchors the model in the specific facility and study question. And it asks the LLM to consult recent literature so that the returned KPIs reflect community practice rather than the modeller's prior habits. The response is constrained to a fixed three-row schema. Forcing the structure has two practical effects. First, the output is directly comparable across projects and across language models. Second, it draws attention to whichever cell is weakest, which is almost always *Scope and boundaries*, and lets that conversation happen up front rather than after a two-hundred-line script has already been written against an unstated assumption. The caveat that Sun et al. raised about LLM ideation in general applies here (C. Si and D. Yang and T. Hashimoto 2024). The model is recombining published practice, not inventing a new study design. The structured schema is what turns that recombination into a usable artefact.

The three rows of the response are not a deliverable in isolation. They are the inputs to the three downstream stages. *Scope and boundaries* determines which resources and processes appear in the SimPy file generated by Creation (Section 3). If the row excludes ambulance routing, no SimPy resource is created for it and no narration mentions it later. The *problem definition* in the first row is the question that Execution (Section 4) must ultimately answer. A trace that does not address it has run, but has not produced evidence. The *KPIs* in the third row become the success criteria of Experimentation (Section 5) and the targets the cross-model benchmark of Evaluation (Section 6) is judged against.

3 STAGE 1: CREATION

The Creation stage takes the three-row brief and produces a runnable, validated SimPy file. It contains three rising-complexity LLM contributions: *Input Modelling*, which turns the *Scope* row into the random-variate generators the model will need; *Model Generation*, which uses a structured prompt with a five-check validation gate to emit a runnable SimPy file; and *Agentic Construction*, which exposes the simulation

[2]Specifically, claude-opus-4-7. The empirical in-simulation comparison of Section 6 additionally exercises GPT-5.1 and Gemini-2.5-flash.

PROMPT

I am planning a discrete-event simulation of an emergency department. Help me formulate the study as a structured problem statement.

NOTES small community hospital, two triage tiers, the team wants to evaluate staffing and routing rule changes.

SEARCH recent literature (2023–2025) on emergency-department simulation studies and pull common KPIs.

RETURN a single table with three rows: *Problem definition*, *Scope & boundaries*, and *KPIs*. Keep each cell to two or three short sentences.

RESPONSE

	ELEMENT	STATEMENT
1	Problem definition	Reduce length of stay in the emergency department and identify whether triage-rule changes or staffing changes deliver the larger improvement under fluctuating arrival rates.
2	Scope & boundaries	From walk-in arrival to admit-or-discharge. Two triage tiers (urgent, non-urgent), a shared imaging and laboratory pool, and fixed bed pools. Ambulance routing, no-shows, and inpatient bed flow are out of scope.
3	KPIs	Mean and 95th-percentile length of stay; door-to-doctor time per triage tier; bed and imaging utilisation; daily throughput.

Figure 6: Problem-formulation prompt and response: a fixed three-row schema (problem, scope, KPIs) that downstream stages consume unchanged.

primitives as MCP tools and lets the LLM construct the model through typed calls rather than a one-shot file emission.

3.1 Input Modelling

The 2025 edition of this tutorial framed input modelling as one of several activities the LLM could help with, and the recent literature has pushed further in that direction. SimLLM (J.-Q. Chen and K. Zhang and R. Zheng and Y. Zhong 2026) fine-tunes code LLMs specifically on SimPy queueing data, and LLM-driven DES frameworks delegate input fitting to multi-agent pipelines (M. Selak and D. Krechel and A. Ulges and S. Spieckermann and N. Stoehr and A. Loehr 2026). Both are powerful, but both also keep the language model as the final arbiter of the distribution choice. We take a deliberately narrower position. Goodness-of-fit is a solved statistical problem. Distribution-free tests have been understood since Anderson and Darling (1952) and packaged in `scipy.stats` for two decades (Delignette-Muller and Dutang 2015). An LLM that picks the distribution by inspection is throwing away that work and adding a non-reproducible step in its place. We use the LLM only at the two ends of the pipeline, proposing candidates and writing the verdict, and give the test the final word in between.

The pipeline is short. The user supplies a numeric column or timestamps. After basic checks (sample size at least 30, no negative values, zeros flagged) the data are summarised numerically (size, mean, standard deviation, coefficient of variation, skewness). The LLM is prompted with that summary and is asked for two or three candidate continuous distributions, one rationale each, and *no fit*. The candidates are then fitted with `scipy.stats`, and Kolmogorov–Smirnov, Anderson–Darling, and a binned chi-square test are run against the fitted CDFs. Figure 7 shows the result on a deliberately bursty arrival trace where all three of the LLM's candidates are rejected at the 5% level. The empirical histogram and the LLM proposals look reasonable in isolation, but the Q-Q plot and the test column make the failure visible. The data are non-stationary (rush hours mixed with off-peak), and no single marginal distribution can absorb that. The verdict at the bottom of the figure is what the LLM writes after reading the test output. The recommendation it produces, refit per hour-of-day, is what the modeller would have reached with classical input-modelling practice. The point is not that the LLM is wrong, but that the test is what made the failure visible.

The artefact produced by input modelling is a single Python expression: a callable that returns one inter-arrival sample, together with a banner noting whether the test accepted the fit or whether the closest-rejected candidate was used as a fallback. The model-generation step (Section 3.2) consumes that expression as the arrival generator. If the banner reports a rejection, the prompt is instructed to refuse the global fit and to ask for hour-of-day stratification instead. The connectivity is therefore mechanical, not narrative.

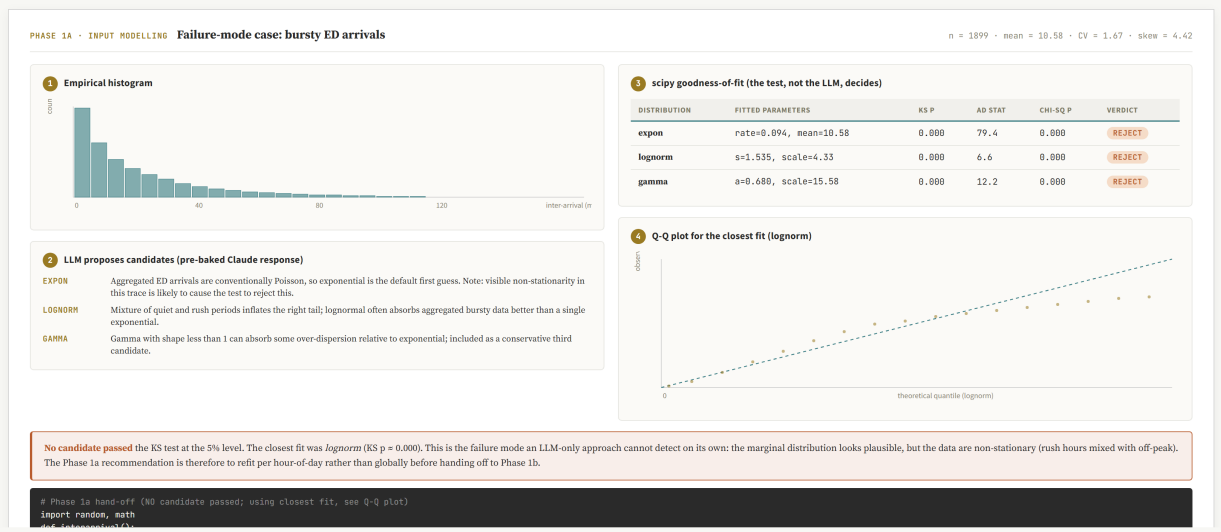


Figure 7: Input modelling on a deliberately bursty ED arrival trace. The LLM proposes three candidate distributions with one-line rationales; `scipy.stats` fits each and runs KS, AD, and chi-square goodness-of-fit. All three candidates are rejected at the 5% level. The Q-Q plot shows systematic deviation in the upper tail, and the verdict (written by the LLM after reading the test output) recommends refitting per hour-of-day.

The brief row that asked for "arrival rate" is answered by a Python line that the next step reads, and the goodness-of-fit verdict travels with it.

3.2 Model Generation

Off-the-shelf simulation packages have offered template-based modelling for two decades. Tools such as Simio, Arena, FlexSim, and AnyLogic let the modeller drag a server, a queue, or a routing decision onto a canvas, configure parameters in dialogs, and publish a runnable model without writing code. The template editor enforces invariants by construction. A server has a capacity. A queue feeds exactly one server. A routing decision lists every outgoing arc. The parameter dialog refuses out-of-domain values. The question we ask in this section is whether the same invariants can be enforced when the model is generated as Python code rather than placed on a canvas. The answer in this paper is yes, but only if the invariants are made an explicit automated check rather than left for the modeller's eye.

The 2025 edition introduced a five-part structured prompt for SimPy generation, comprising Role, Goal, Scenario, Inputs, and Output. We retain the structure but feed it from the upstream artefacts. The Role asks the LLM to write as a simulation engineer. The Goal is to produce a runnable SimPy file for the stated problem. The Scenario is the *Scope and boundaries* row, copied verbatim, so the LLM cannot drift into adjacent ED variants the modeller did not ask for. The Inputs section pastes the arrival generator from input modelling together with the fitted distributions for service times. The Output section asks for one Python file with no dependencies beyond SimPy and the standard library. SimLLM (J.-Q. Chen and K. Zhang and R. Zheng and Y. Zhong 2026) has shown that fine-tuned 7B-class models can generate executable SimPy on a similar prompt template, and M. Selak and D. Krechel and A. Ulges and S. Spieckermann and N. Stoehr and A. Loehr (2026) reach the same conclusion via a multi-agent pipeline. Both confirm that frontier-model zero-shot prompting is a defensible default for a tutorial reader.

Frontier LLMs are powerful but probabilistic. On the same prompt, two runs can produce structurally different SimPy files, and silent bugs in the generated code can survive a casual read. We therefore put a fixed five-check validation gate between the generated file and the downstream stages (Figure 8). Process

logic checks that the patient flow matches the brief end to end. Connectivity checks that each producer wires into the declared consumer with no orphan resources. Parallel-versus-sequential checks that a pool with capacity c actually serves c patients in parallel. This is one of the silent failure modes that motivated the WSC 2025 lessons-learned review of this case study. A single per-pool "manager" process collapses effective parallelism to one even when `simpy.Resource(capacity=c)` is declared correctly. Distribution checks verify that sampled service times match the input-modelling fit in mean, variance, and support. Theory checks compare realised utilisation, mean queue length, and mean waiting time against closed-form Erlang-C and Allen–Cunneen predictions for the matching $M/G/c$ system. If any check fails, the failing check is appended to the next prompt verbatim and the LLM is asked to regenerate only the affected section. If three rounds of regeneration do not pass, the case is handed off to the agentic-construction route of Section 3.3. The artefact that leaves Creation is therefore not just a SimPy file. It is a SimPy file plus a passing validation report. Either both travel forward, or neither does.

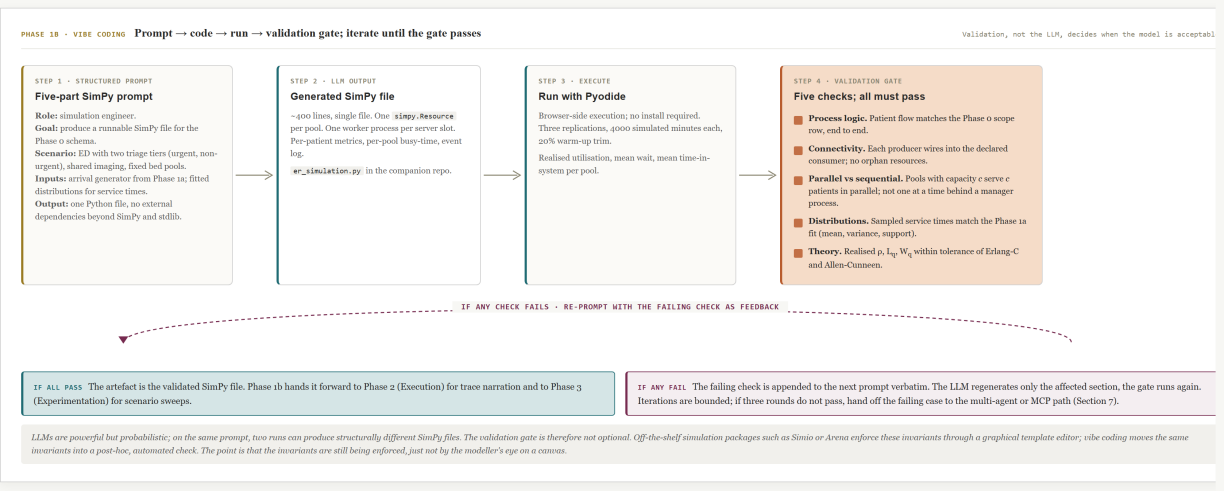


Figure 8: Vibe-coding loop with a five-check validation gate (process logic, connectivity, parallelism, distributions, theory). If any check fails, the failing check is appended to the next prompt and iterations are bounded. Off-the-shelf template editors enforce the same invariants at draw time; the gate is the code-based equivalent.

The validated SimPy file is published as `phase1b/er_simulation.py` in the companion repository (Section 6). Execution (Section 4) reads it as input. Experimentation (Section 5) runs scenario sweeps against it and scores them against the KPI list.

3.3 Agentic Construction

The validation gate of Section 3.2 catches model files that fail invariants the LLM was asked to respect. It does not change the failure mode itself. The LLM produces a complete file in one shot, the gate either passes it or rejects it, and a rejection sends the entire file back for regeneration. This is acceptable for tutorial-scale models, but it scales poorly. A failure on one resource regenerates the whole file. A small change to the brief regenerates the whole file. The diagnostic information is the failed check, not the offending line. This subsection asks whether a different interaction pattern, in which the LLM is constrained to construct the model through a small set of typed actions rather than emit a Python file, gives back what one-shot vibe coding loses.

The Model Context Protocol is an open standard, introduced by Anthropic in November 2024, for connecting language models to external tools and data sources (Anthropic 2024). An MCP *server* exposes three primitives: tools (functions the LLM can call), resources (read-only data the LLM can request), and

prompts (parameterised templates). An MCP *client*, embedded in the LLM host application, discovers the server's capabilities at session start and routes the model's tool calls to the server over JSON-RPC. The protocol has been the subject of two recent surveys, one focused on MCP itself (A. Singh and A. Ehtesham and S. Kumar and T. Talaei Khoei 2025) and one positioning it within the broader landscape of agent interoperability protocols (A. Ehtesham and A. Singh and G. K. Gupta and S. Kumar 2025).

The relevance to simulation is direct. A SimPy model is a small set of declarations and a small set of process definitions. If those operations are exposed as MCP tools, the LLM no longer writes a Python file. It calls `define_resource`, `define_process`, `wire`, `run`, and `query_kpis` in sequence, and the server enforces the invariants on each call. A capacity argument that is not an integer is rejected at the call. A wire that points at an undeclared consumer is rejected at the call. Theory checks run after `run` and report a diagnostic that the LLM can read and act on without regenerating any code. The construction of the model becomes a structured dialogue with a typed API, rather than a bet on a one-shot Python emission.

The companion repository ships a minimal `simpy-mcp` server that exposes five tools: `define_resource(name, capacity)`, `define_process(name, generator_body)`, `wire(producer, consumer)`, `run(duration, replications, seed)`, and `query_kpis(group_by=...)`. Construction of the ED model proceeds by sequence. The LLM reads the brief, calls `define_resource` for triage, imaging, lab, and beds, calls `define_process` for arrivals, triage, and admit-or-discharge, and `wires` the producers to the consumers. `run` returns a structured result. `query_kpis` produces the rows the brief asked for. If a wire is wrong, the server returns a typed error and the LLM corrects the call. If a theory check disagrees with the realised utilisation, the server returns the disagreement and the LLM revises only the relevant parameter.

Vibe coding, with the validation gate of Section 3.2, is the right path when the modeller wants a single file they can read, version, and run outside the loop. The output is a self-contained `.py` that anyone with SimPy installed can execute. The MCP path is the right path when the model is built incrementally, when invariants must be enforced at construction time rather than after the fact, or when the same brief will be re-run against shifting parameter sets. The two paths are not in opposition. The companion repository ships both, and the framework figure (Figure 5) shows them as alternative routes through the Creation stage. Recent work in this direction independently confirms the trade-off: Vasa and Sarjoughian (2025) take the structured construction route via PDEVS-LLM with a statechart-driven workflow, and J. Kleiman and K. Frank and S. Campagna (2025) present a general simulation-agent framework that decouples natural-language scenario configuration from model execution. Our contribution is the side-by-side: the same brief, the same KPIs, two routes, and a comparison whose dimensions are agreed in advance (iteration count, first-pass executability, diagnosability of failures).

4 STAGE 2: EXECUTION

The Execution stage takes the validated SimPy file and runs it. It contains three rising-complexity LLM contributions: *Narration* translates the raw trace into entity-grouped plain English; *Orchestration* lets the LLM coordinate run-time activity across processes; and *Decision Loop* pauses the simulation at every decision point and lets the LLM choose, with a comparison against fixed heuristic baselines.

4.1 Narration

The default SimPy trace is a stream of `(time, entity, event)` tuples. It is unreadable in raw form. The 2025 edition introduced LLM-driven trace narration, prompting the model with a sliding window of trace lines and asking for a short paragraph of plain English. The same approach carries over for the 2026 revision, with two refinements. First, the prompt now includes the brief row that the trace must address, so the narration is purposive rather than a transcript. Second, the narration is grouped by entity (patient) rather than by clock time, which lets the reader follow one patient end to end instead of reconstructing journeys from interleaved events.

4.2 Orchestration

The 2025 edition treated process orchestration as the middle Execution-stage capability: an LLM-mediated coordination of run-time activity across SimPy processes (e.g., adjusting service rates or re-routing entities mid-run in response to a state summary). The 2026 revision retains the placeholder for that material; readers are referred to the 2025 paper (Dehghani et al. 2025) for the full treatment. The new contributions of this paper concentrate on Narration (above) and Decision Loop (below).

4.3 Decision Loop

The ED running example has a recurring decision point. Each arriving patient must be assigned to one of b inpatient beds (or held in queue if all are busy). The simplest heuristic is round-robin: assign cyclically by bed index. A slightly stronger rule prioritises by severity, then breaks ties by time-in-queue. A clinician-trained rule looks at remaining length-of-stay estimates and tries to balance bed turnover against acuity. All three are deterministic functions of the current system state.

We add a fourth option. At each assignment step the simulation pauses, serialises the relevant state (current bed occupancy, severity, time-in-queue, expected length of stay), sends it to an LLM with a short instruction, and uses the returned bed index. Recent work has demonstrated this pattern in adjacent domains. Y. Quan and Z. Liu (2024) place an LLM agent inside a multi-echelon inventory simulation as the order-quantity decider, and He and Bian (2025) place one inside a transportation simulation as the route-choice decider. Pseudocode for our setting is short:

```
def llm_assign(state, llm):
    """Decide which bed index to use, given the current ED state."""
    prompt = render_prompt(state) # serialise occupancy, severity, queue
    reply = llm(prompt, temperature=0.0, max_tokens=8)
    return parse_bed_index(reply) # int in [0, b)

# Drop-in: the SimPy process calls llm_assign(state, llm) at each
# admission step instead of the round-robin or severity-priority rule.
```

The Execution stage is therefore a place where the simulation engine and the language model trade control. At every decision point the engine hands over the state, the model returns a choice, and the engine resumes. Memory is what turns a sequence of independent calls into a coherent policy. The companion implementation keeps a rolling ring of the five most recent decisions, plus a one-line "policy so far" summary that the LLM compacts every ten calls, and appends both to the next prompt alongside the live state. The mechanism is light enough to be written in roughly thirty lines of Python, runs without any agent-orchestration framework, and is portable to the browser via Pyodide. Three operational properties follow. First, latency matters. A frontier-model call adds hundreds of milliseconds to seconds of wall-time per decision, which is not free in a study with 10^5 assignments. The companion application caches identical state vectors and batches calls in trace replay so that the cost stays within tutorial budgets. Second, reproducibility matters. We fix temperature to zero and record the prompt-and-reply for every decision so the run can be replayed. Third, the comparison is the point. Running the same trace through FIFO, severity-priority, and the LLM gives a row-by-row KPI table judged against the same brief. The cross-model results, including the agreement-rate diagnostic between LLM and the matching heuristic, are reported in Section 6 as part of the Evaluation stage.

For comparability across vendors, the Execution stage keeps a small driver that abstracts the model call into a uniform interface. The same loop runs with Claude, Gemini, or GPT behind it. The companion application exposes the four heuristics as built-in baselines. To exercise the LLM-in-the-loop variant the user supplies their own API key for whichever vendor they prefer. We did not embed shared keys in the public site for cost reasons. The keys never leave the browser tab. The point of the section, however, is

methodological rather than UI-shaped. A simulation engine that can call out to a language model at decision time is a different artefact from one whose decision rule is fixed at compile time, and the methodological consequences (latency, cost, reproducibility, agreement-rate diagnostics) are what we care about.

5 STAGE 3: EXPERIMENTATION

The Experimentation stage takes the validated model and turns its outputs into something a reader can act on. It contains three rising-complexity LLM contributions: *Summary Generation* (figure selection plus a short written interpretation of results); *Scenario Ideation* (proposing factors and levels from the brief); and *Optimisation* (using the LLM to drive parameter search, with explicit caveats).

5.1 Summary Generation

The simplest Experimentation contribution combines visualisation and narrative interpretation. SimPy logs are amenable to direct plotting: the companion artefact ships a small `viz.py` that reads the standard log format and emits matplotlib figures (KPI bar charts across scenarios, queue-length time series with confidence bands, utilisation heatmaps by resource and shift, and length-of-stay distributions by severity). The role of the LLM here is figure selection rather than figure drawing. Given the KPI list and the scenario table, it is asked which two or three plots best support the comparison the brief calls for. Because the plots are produced by code rather than by the LLM, what the LLM actually emits is a sequence of `viz.py` function calls. The figures themselves are reproducible from the log file alone.

The narrative half is a short written interpretation. The 2025 edition used a third-party tool (Napkin) to generate visual summaries from log text. The 2026 revision keeps the spirit (LLM as the writer of the report) but moves the activity inside the workflow. The LLM is given three things: the KPI rows from the brief, the scenario table, and short captions for the plots. It is asked to write a paragraph addressed to a stakeholder, naming which scenario performs best on which KPI and where the trade-offs lie. The output is reviewed by the modeller before it leaves the loop. The reason for keeping a human in this seat is the same as in input modelling. The model is good at recombining what it sees, and that is exactly the right capability for synthesising a written summary, but the responsibility for the claim rests with the modeller. Recent work on LLM-assisted result interpretation in adjacent domains supports this division of labour, with the model handling first-draft narrative and the analyst keeping the final word (M. Selak and D. Krechel and A. Ulges and S. Spieckermann and N. Stoehr and A. Loehr 2026; J. Kleiman and K. Frank and S. Campagna 2025).

5.2 Scenario Ideation

The factors and levels are not invented from scratch. They are read off the brief. The KPI row tells us what to measure (e.g., mean length of stay, 95th-percentile waiting time, bed-utilisation). The scope row tells us what is movable in the system under study (e.g., bed count, triage staffing, hour-of-day arrival pattern). A small full-factorial or fractional-factorial design over the movable factors, replicated to the precision the KPI row demands, produces the data table. The LLM contribution at this stage is bounded and explicit. It proposes a starting design from the brief, the modeller accepts or trims it, and the rest is mechanical: generate the scenario file, run, collect.

5.3 Optimisation

Optimisation is the most complex Experimentation capability we attempt with an LLM in the loop, and the one we have the least to claim. A frontier LLM can suggest the next scenario to run given results so far — a loose Bayesian sequential design pattern that converges if the response surface is well behaved. It is not, however, a substitute for a real simulation optimiser such as OptQuest, nor for a black-box optimiser like SMAC. Two failure modes are worth naming. The model has no automatic exploration-versus-exploitation

budget; without an explicit instruction, it tends to exploit. And it has no systematic way to reason about variance reduction; it does not know that a doubled replication count quarters the standard error. We therefore use the LLM-driven optimiser only on small designs with cheap evaluations, and report the search trace alongside the optimum so the reader can judge.

6 STAGE 4: EVALUATION

The Evaluation stage closes the workflow with three rising-complexity LLM contributions, each grounded in artefacts the prior stages have already produced:

- **Reproducibility.** Every figure, every prompt, and every result is regenerable from the public repository, and the live in-browser site lets a reader exercise the workflow without installing anything.
- **Validation.** Realised KPIs are cross-checked against closed-form $M/G/c$ bounds (the harness from the validation gate of Section 3.2) and against an off-the-shelf commercial reference model (Arena or Simio configured to the same brief), to provide a second-source check that does not depend on the SimPy implementation.
- **Cross-Model Benchmark.** The Decision Loop of Section 4.3 is exercised through multiple frontier LLMs on the same paired-seed harness, producing a uniform scorecard.

Reproducibility. Every artefact in this paper is reproducible from the public repository introduced at the end of the introduction. The repository hosts the validated SimPy file (`phase1b/er_simulation.py`), the input-modelling demos (clean and bursty), the prompts used at each stage, the validation harness, the `simpy-mcp` server, and the comparison-table run logs. The companion live site loads SciPy and NumPy in the browser via Pyodide, so the input-modelling pipeline (candidates from the LLM, fits and goodness-of-fit tests from `scipy.stats`, Q-Q plot, verdict, copy-pasteable SimPy line) runs end to end without a server. Tabs are organised by the five stages of Figure 5, plus a tab that lets the reader supply their own API key to exercise the LLM-in-the-loop variants of Section 4.3.

Validation. KPI claims throughout the paper are cross-checked against analytical $M/G/c$ bounds (Erlang-C and Allen–Cunneen for the matching multi-server model); the harness that performs the comparison is in `phase1b/` and is the same harness used by the validation gate of Section 3.2. As a second-source check we additionally reproduce the brief in an off-the-shelf commercial package (Arena or Simio) configured to the same arrival distribution, service-time fits, and resource pools, and report the realised KPIs side by side with the SimPy run.

Cross-Model Benchmark. The most readily measurable claim in the paper is the LLM-in-the-loop variant of Section 4.3. We ran the ED simulation across four admission policies on the same eight paired seeds: *FIFO* (oldest queued patient first), *severity-priority* (lowest severity number first, ties by arrival time), *GPT-5.1 with rolling memory*, and *Gemini-2.5-flash with rolling memory*. Each replication generates 25 patients across 4 beds with severity-stratified log-normal length-of-stay, producing 7–20 admission decisions per replication and 200 LLM calls per model in total. The LLM policies receive at every decision: the live ED state (bed occupancies, expected releases), the queue (each patient with severity and time-in-queue), the last five decisions, and a one-line "policy so far" summary that the LLM compacts every ten calls. Temperature is fixed to zero, so the only randomness across replications is the simulation seed.

Three patterns deserve attention. First, FIFO and severity-priority differ along the expected axis: severity-priority more than halves urgent-patient wait (from 27.9 min to 12.9 min) at the cost of doubling routine-patient wait. The mean wait alone hides this trade-off completely. Second, GPT-5.1 with memory reproduces severity-priority *exactly*: 0 of 200 decisions deviated from the rule, even though the prompt explicitly framed the trade-off rather than naming a target. The agreement-rate diagnostic of Section 4.3 did its job: where the model agrees with the heuristic, the model is doing what a sensible rule would do, at a wall-time cost of about 770 ms per decision. Third, Gemini-2.5-flash with memory deviates on 16.5%

Table 1: Admission-policy benchmark on the ED running example: N=8 paired replications, 25 patients each, B=4 beds. All wait times in minutes, mean $\pm$ std across replications.

Policy	Mean wait	p95 wait	Sev-1 wait	Sev-3 wait	Util.	Decision lat.
FIFO	28.2 ± 21.2	71.5 ± 33.4	27.9 ± 23.5	30.0 ± 30.6	0.83 ± 0.08	~ 0 ms
Severity-priority	33.0 ± 27.3	112.3 ± 79.0	12.9 ± 8.9	64.1 ± 76.5	0.85 ± 0.09	~ 0 ms
GPT-5.1 + memory	33.0 ± 27.3	112.3 ± 79.0	12.9 ± 8.9	64.1 ± 76.5	0.85 ± 0.09	773 ms
Gemini-2.5-flash + mem.	31.7 ± 24.7	93.5 ± 54.2	12.9 ± 9.0	48.1 ± 50.6	0.85 ± 0.09	585 ms

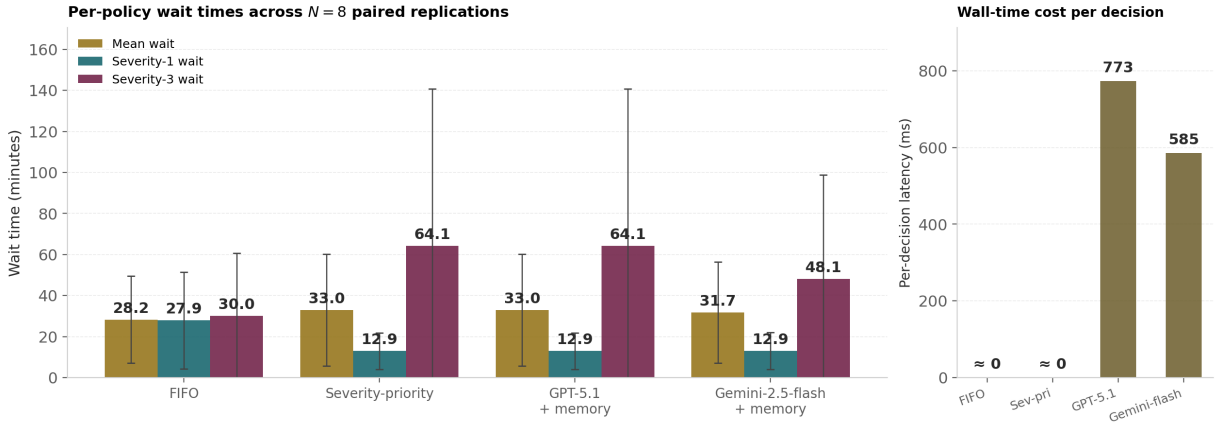


Figure 9: Results of Table 1 as a bar chart, rendered directly from `summary.json`. Left: per-policy wait times with ± 1 std error bars. Right: wall-time cost per admission decision. GPT-5.1 with memory matches severity-priority bar-for-bar; Gemini-2.5-flash with memory drops Sev-3 wait from 64.1 to 48.1 minutes while keeping Sev-1 wait constant.

of decisions (33 of 200), and its deviations are not random. They cluster on the longest replications, where queues lengthen and routine patients risk starvation. On replication 1, Gemini interleaves moderate and routine patients with the urgent ones and cuts routine wait from 218 to 105 minutes while keeping urgent wait essentially unchanged (18.1 min vs 18.5). This is the kind of contextual hybridisation that a static rule cannot express, and is exactly the value proposition for an LLM-in-the-loop variant.

The cost of that value is non-trivial. A 600–800 ms decision latency adds up across ~ 100 admission decisions per simulated replication, and the LLM policies were 10^5 times slower per decision than either heuristic. For a single tutorial-scale model this is acceptable. For a study with millions of decisions per scenario it is not. The companion repository ships a caching layer (identical state vectors return their prior decision) and a batched-replay mode for offline scenario sweeps; both reduce effective latency to the order of the heuristics. The methodological point of this subsection, however, stands without those engineering details: a memory-aware LLM-in-the-loop is a genuinely different artefact from a fixed rule, and whether the difference is worth its cost is a property of the study, not of the modelling stack.

Discussion. This tutorial has organised GenAI-assisted simulation work into a 5 by 3 capability ladder with typed hand-offs and a validation gate, and has used a single emergency-department case study to walk it end to end. The argument throughout has been that LLMs are best deployed at the boundaries of each stage, where they propose, narrate, and synthesise, and that the substantive decisions inside each stage are best left to instruments that already exist for that purpose. Goodness-of-fit tests pick the input distribution. Closed-form Erlang-C bounds judge the model. Mechanical sweeps generate the scenarios. In each case the model is the writer, not the arbiter. Three concrete advances over the 2025 edition stand out. The first is reproducibility: every figure, every prompt, and every result in this paper is regenerable from

the public repository, and the live in-browser site lets a reader exercise the workflow without installing anything. The second is the agentic construction route of Section 3.3, which replaces one-shot vibe coding with structured construction through a typed tool catalogue, and which is shown side by side with vibe coding on the same brief. The third is measurement: the same validation gate that protects the workflow also produces a uniform scorecard for comparing frontier LLMs, and the pilot benchmark of Section 6 reports a first set of numbers on the ED case. None of these is a finished result, and the companion repository is structured so that readers can extend the comparison to their own briefs and their own models.

ACKNOWLEDGMENTS

REFERENCES

Anderson, T. W., and D. A. Darling. 1952. "Asymptotic Theory of Certain "Goodness of Fit" Criteria Based on Stochastic Processes". *Annals of Mathematical Statistics* 23(2):193–212.

Anthropic 2024, November. "Introducing the Model Context Protocol". Anthropic Engineering Blog. Accessed April 2026.

J.-Q. Chen and K. Zhang and R. Zheng and Y. Zhong 2026. "SimLLM: Fine-Tuning Code LLMs for SimPy-Based Queueing System Simulation". arXiv preprint arXiv:2601.06543.

Dehghani, M., S. Belsare, and N. Sadeghi. 2025. "A Tutorial on Generative AI and Simulation Modeling Integration". In *Proceedings of the 2025 Winter Simulation Conference*, 1197–1211. Seattle, WA.

Delignette-Muller, M.-L., and C. Dutang. 2015. "`fitdistrplus`: An R Package for Fitting Distributions". *Journal of Statistical Software* 64(4):1–34.

A. Ehtesham and A. Singh and G. K. Gupta and S. Kumar 2025. "A Survey of Agent Interoperability Protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP)". arXiv preprint arXiv:2505.02279.

He, P., and W. Bian. 2025. "Agentic Large Language Models for Day-to-Day Route Choices". *Transportation Research Part C: Emerging Technologies* 178:105311.

K. Hu 2023, February. "ChatGPT Sets Record for Fastest-Growing User Base—Analyst Note". Reuters. Accessed April 2026.

J. Kleiman and K. Frank and S. Campagna 2025. "Simulation Agent: A Framework for Integrating Simulation and Large Language Models for Enhanced Decision-Making". arXiv preprint arXiv:2505.13761.

Y. Quan and Z. Liu 2024. "InvAgent: A Large Language Model based Multi-Agent System for Inventory Management in Supply Chains". arXiv preprint arXiv:2407.11384.

M. Selak and D. Krechel and A. Ulges and S. Spieckermann and N. Stoehr and A. Loehr 2026. "Agentic Insight Generation in VSM Simulations". arXiv preprint arXiv:2604.12421.

C. Si and D. Yang and T. Hashimoto 2024. "Can LLMs Generate Novel Research Ideas? A Large-Scale Human Study with 100+ NLP Researchers". arXiv preprint arXiv:2409.04109.

A. Singh and A. Ehtesham and S. Kumar and T. Talaei Khoei 2025. "A Survey of the Model Context Protocol (MCP): Standardizing Context to Enhance Large Language Models". Preprints.org 202504.0245.

Vasa, V. K. S., and H. S. Sarjoughian. 2025. "Generative Statecharts-Driven PDEVS Modeling". In *Proceedings of the 2025 Winter Simulation Conference*. Code at https://github.com/comses/PDLM.

Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, *et al*. 2017. "Attention Is All You Need". In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 5998–6008.

M. Zao-Sanders 2025, April. "How People Are Really Using GenAI in 2025". Harvard Business Review. Accessed April 2026.

AUTHOR BIOGRAPHIES

MOHAMMAD DEHGHANIMOHAMMADABADI is an Associate Teaching Professor in the Department of Mechanical and Industrial Engineering at Northeastern University. His research focuses on developing intelligent simulation frameworks integrated with optimization and AI-based models to support

decision-making, with applications in healthcare, manufacturing, and supply chain. His email address is m.dehghani@northeastern.edu.

SAHIL BELSARE is a simulation scientist with expertise in digital twin technologies and simulation modeling. He applies machine learning models to enhance decision-making in complex systems, focusing on optimization and reinforcement learning integrated with simulation platforms. His email is belsare.s@northeastern.edu.

NEGAR SADEGHI is a Ph.D. student in Industrial Engineering at Northeastern University. Her research focuses on developing intelligent simulation models for pharmaceutical supply chains and production-distribution systems. Her email address is sadeghi.ne@northeastern.edu.